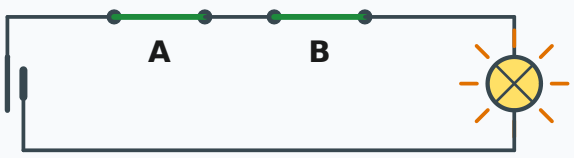
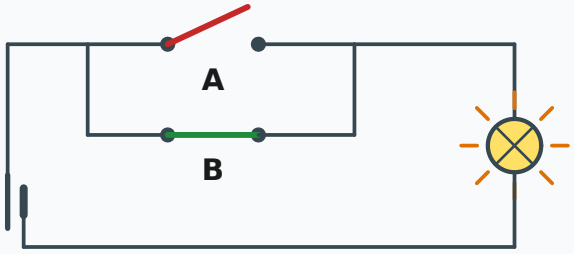
Chapter 7

Elementary logic

true or false, wired up like light switches



both switches on: lamp on



either switch on: lamp on

Logic is the maths of **true** and **false**. It's how computers make decisions (every if in a program), how search engines combine words, and how you check whether an argument actually holds up.

There are only a handful of ideas: **not**, **and**, **or**, and **if... then**. We'll wire each one as a light switch circuit, then write it as a **truth table** (a list of every possible case), and finally as Python code.

Not on a lecture slide, but underneath Chapter 6 (sets) and every program you'll write.

Statements: sentences that are true or false

A **statement** is a sentence that is definitely **true** or definitely **false**, even if you don't know which yet. Which of these are statements?

"Paris is in France."	✓	*statement: true*
"7 is an even number."	✓	*statement: false (still a statement!)*
"Shut the door."	✗	*an order: can't be true or false*
"Is it raining?"	✗	*a question: not a statement*
"x > 3"	?	*depends on x: a statement once x has a value*

Being true or false is called the statement's **truth value**, written **T** or **F**. Computers write 1 and 0, or True and False. We'll use letters like P and Q as names for statements, just as x names a number.

NOT: flip it

RULE: NOT

$\neg P$ is true when P is false, and false when P is true.

Two NOTs cancel: $\neg\neg P$ has the same value as P.

NOT turns true into false and false into true, like flicking a switch. If P is "it is raining", then $\neg P$ ("not P") is "it is not raining".

P	$\neg P$
T	F
F	T

Only 2 rows: P can only be T or F.

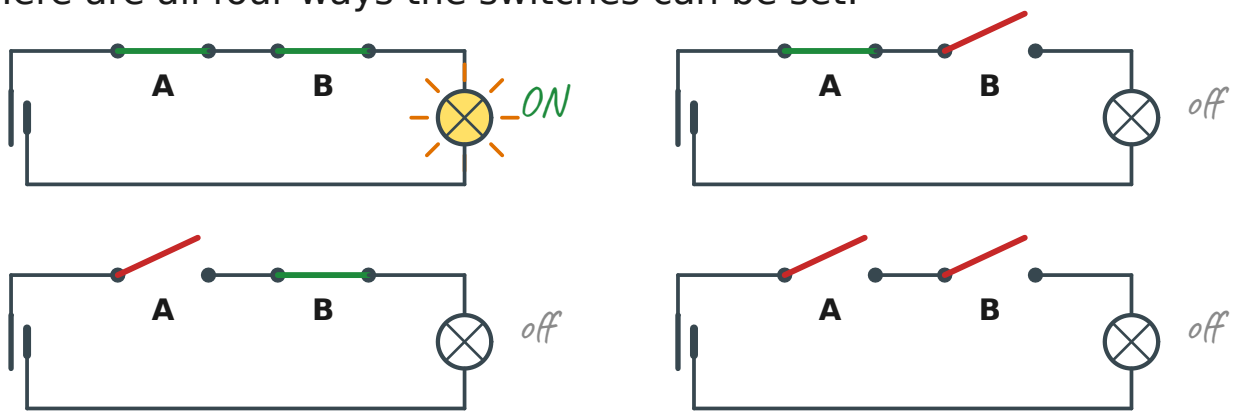
NOT just swaps them.

Set version (Chapter 6): the complement A'

Other ways books write it: ~P, P̄, !P (in C and Java), not P (Python).

AND: two switches in a row

Wire two switches one after the other ("in series"). Electricity must get through **both**, so the lamp lights only when A **and** B are both on. Here are all four ways the switches can be set:



Replace "on" by T and "off" by F, and you have the **truth table** for AND. We write "A and B" as $A \wedge B$:

A	B	$A \wedge B$
T	T	T
T	F	F
F	T	F
F	F	F

just one T in the answer column: the row where both are T

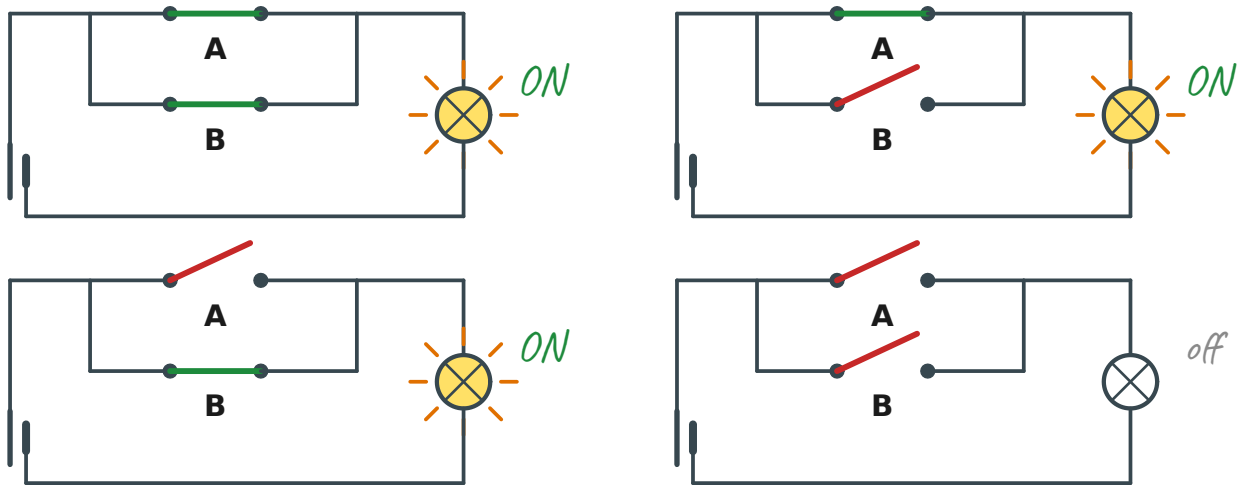
$\wedge$ looks like the $\cap$ of sets: $A \cap B$ = in A AND in B

RULE: AND

$A \wedge B$ is true **only** when A and B are both true. One F anywhere makes it F.

OR: two switches side by side

Now wire the switches side by side ("in parallel"). Electricity can take **either** path, so the lamp lights if A is on, or B is on, **or both**.



A	B	$A \vee B$
T	T	T
T	F	T
F	T	T
F	F	F

just one F: the row where both are off

∨ is like the ∪ of sets: A ∪ B = in A OR in B (or both)

RULE: OR

$A \vee B$ is false **only** when A and B are both false. One T anywhere makes it T.

Maths "or" includes both. In everyday speech "or" often means one but not the other. Maths has a separate name for that, on the next page.

Exclusive or: the landing light

A menu says "tea **or** coffee included". You can't have both! That's **exclusive or** (XOR, written $\oplus$): true when exactly one is true.

A	B	$A \oplus B$
T	T	F
T	F	T
F	T	T
F	F	F

The light on a staircase has a switch at the bottom and one at the top. Flip either one and the light changes. Light on = the two switches are in different positions: that's XOR.

XOR is everywhere in computing: it's how a computer adds two binary digits (Chapter 10) without the carry, and a basic tool in encryption.

How many rows, and in what order

With one letter there are 2 cases, with two letters 4. Every new letter doubles the count, because each old row splits into a T version and an F version.

RULE: rows of a truth table

n different letters give 2^n rows: 1 letter 2 rows, 2 letters 4, 3 letters 8, 4 letters 16.

Standard order: the **first** column is the top half T, the bottom half F (TTTTFFFF for 3 letters).

Each next column halves the blocks: TTFFTTFF, then TFTFTFTF. The **last** letter column always alternates T, F.

This order lists every case exactly once, and makes your table match everyone else's.

RULE: order of operations (precedence)

$\neg$ first, then $\wedge$, then $\vee$, then $\Rightarrow$, then $\Leftrightarrow$. Brackets override all of these.

Like BODMAS: $\neg$ behaves like a minus sign, $\wedge$ like $\times$, $\vee$ like $+$.

$\neg P \vee Q \wedge R$ means $(\neg P) \vee (Q \wedge R)$; $P \wedge Q \Rightarrow R$ means $(P \wedge Q) \Rightarrow R$.

PROCEDURE: building a truth table

1. Count the letters n; draw 2^n rows.
2. Fill the letter columns in the standard order above.
3. Add one column for each piece, innermost brackets first, following the precedence.
4. The last column is the answer for the whole statement.

Example. Build the table for $(P \wedge Q) \vee \neg R$. Three letters, so $2^3 = 8$ rows:

P	Q	R	$P \wedge Q$	$\neg R$	answer
T	T	T	T	F	T
T	T	F	T	T	T
T	F	T	F	F	F
T	F	F	F	T	T
F	T	T	F	F	F
F	T	F	F	T	T
F	F	T	F	F	F
F	F	F	F	T	T

P: TTTTFFFF
Q: TTFFTTFF
R: TFTFTFTF

answer = (P∧Q) ∨ ¬R
F only when R is T
and P∧Q is F

Building bigger tables, column by column

Is "not (A and B)" the same as "(not A) or (not B)"? Don't guess: build a table. Add one column for each small piece, then combine:

A	B	$A \wedge B$	$\neg(A \wedge B)$	$\neg A$	$\neg B$	$\neg A \vee \neg B$
T	T	T	F	F	F	F
T	F	F	T	F	T	T
F	T	F	T	T	F	T
F	F	F	T	T	T	T

The two outlined columns match in every row, so the statements always agree. This is **De Morgan's law**. In words: "it's not true that both happened" means "at least one of them didn't happen".

$$\neg(A \wedge B) = \neg A \vee \neg B \qquad \neg(A \vee B) = \neg A \wedge \neg B$$

NOT flips AND into OR... *...and OR into AND*

Programming use: "not (logged_in and paid)" is the same test as "not logged_in or not paid". Handy when tidying up conditions.

DEFINITION: equivalence, tautology, contingency, contradiction

Two statements are **logically equivalent** ($\equiv$) when their final columns match in every row.

A **tautology** is true in every row, whatever the letters are. Example: $A \vee \neg A$.

A **contradiction** is false in every row. Example: $A \wedge \neg A$.

A **contingency** is neither: the final column has at least one T and at least one F. Most everyday statements are contingencies ("it rains today").

De Morgan: $\neg(A \wedge B) \equiv \neg A \vee \neg B$ and $\neg(A \vee B) \equiv \neg A \wedge \neg B$. NOT goes inside and swaps $\wedge$ with $\vee$.

Example. One letter, so 2 rows:

A	$\neg A$	$A \vee \neg A$	$A \wedge \neg A$	A
T	F	T	F	T
F	T	T	F	F

all T: tautology

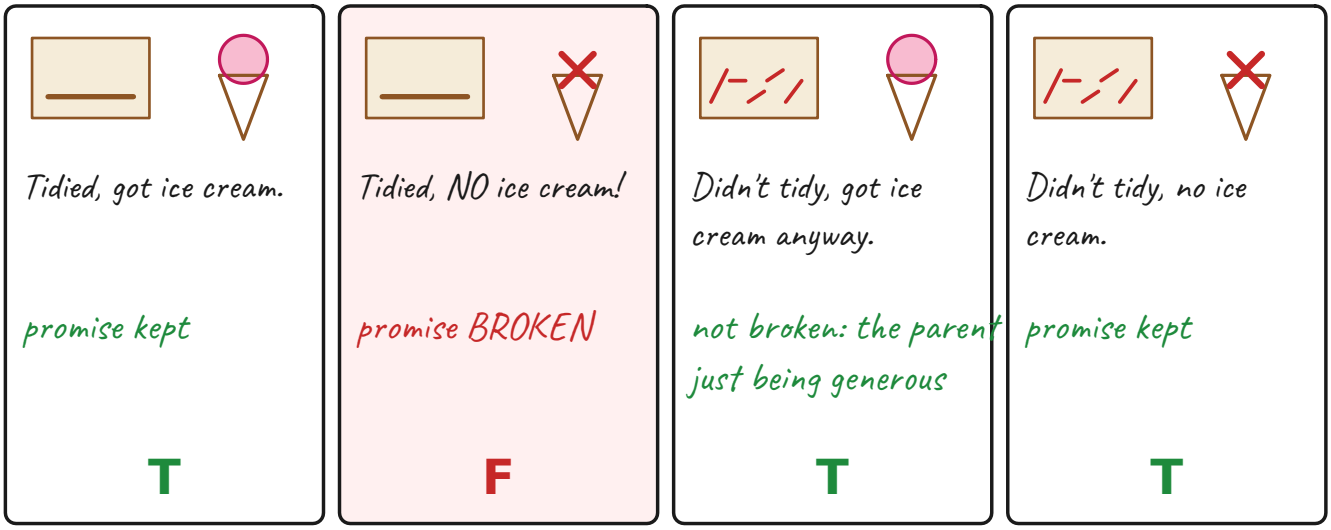
all F: contradiction

mix of T/F: contingency

Exam habit: build the truth table, then classify the final column: all T / all F / mixed.

IF... THEN: a parent's promise

A parent says: **"If you tidy your room, then you get ice cream."** When has the parent **broken** the promise? Look at the four things that can happen:



Only panel 2 breaks the promise. So "if P then Q", written $P \Rightarrow Q$, is **false only when P is true and Q is false**. Everything else counts as true:

P	Q	$P \Rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

P = "you tidy", Q = "you get ice cream"

rows 3 and 4 feel odd, but the promise said nothing about what happens if you DON'T tidy, so it can't be broken there

RULE: implication

$P \Rightarrow Q$ is false **only** when P is T and Q is F.

$P \Rightarrow Q \equiv \neg P \vee Q$: "either P doesn't happen, or Q does". Same table, row for row.

Negation: $\neg(P \Rightarrow Q) \equiv P \wedge \neg Q$. To break a promise, P must happen **and** Q must not.

Example. Negate "if you tidy, you get ice cream":

$$\neg(T \Rightarrow I) \equiv T \wedge \neg I \longleftarrow \text{"you tidied and you did not get ice cream"}$$

✗ Common mistakes

✗ $\neg(P \Rightarrow Q) \equiv \neg P \Rightarrow \neg Q$ ✓ $\neg(P \Rightarrow Q) \equiv P \wedge \neg Q$

the negation of an "if" is not another "if"

Turning an "if" around

Start with a true statement: **"If it's a cat, then it's an animal."** Now shuffle it in three ways. Which versions are still true?

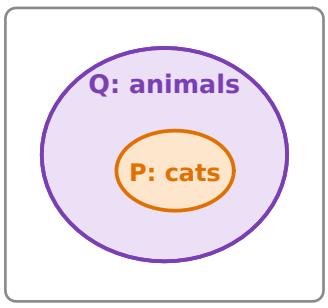
original	If it's a cat, then it's an animal. *true*
converse (swap)	If it's an animal, then it's a cat. *FALSE: a dog is an animal*
contrapositive	If it's not an animal, then it's not a cat. *true: swap AND put NOT on both; always matches the original*
inverse	If it's not a cat, then it's not an animal. *FALSE: a dog again (NOT on both, no swap)*

The picture that settles it. Draw everything that makes P true as a region, and the same for Q. "If P then Q" says **the P region sits entirely inside the Q region**.

✎ YOUR GO (pencil first)

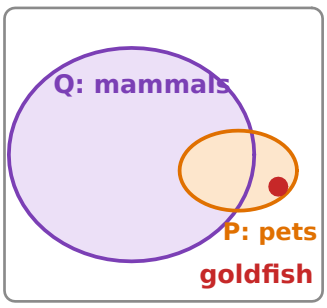
Picture a small blob "cats" inside a big blob "animals". Where would a dot have to be to break "if it's a cat, then it's an animal"? Is there any room for it?

if cat, then animal



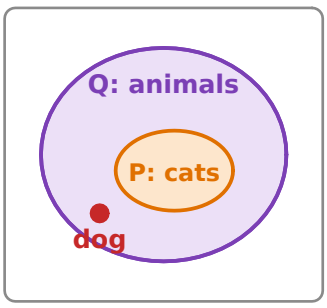
true: every cat is an animal

if pet, then mammal



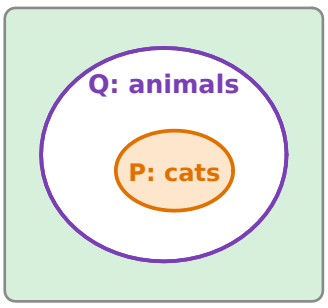
FALSE: a goldfish is a pet, not a mammal

converse: if animal, then cat



FALSE: a dog is an animal, not a cat

contrapositive: if not animal, then not cat



true: nothing outside Q is inside P

A **counterexample** to "if P then Q" is a dot **inside P but outside Q**. When P sits wholly inside Q there's nowhere to put one, so the statement is true. The **converse** asks the opposite (is Q inside P?), which is a different question. The **contrapositive** looks at the outside: everything outside Q is automatically outside P, the same picture read from the other side.

When a statement and its converse are **both** true, we say "P **if and only if** Q", written $P \Leftrightarrow Q$. Example: "a number is even if and only if it divides by 2". The two sides always stand or fall together.

A very common reasoning error is assuming the converse: "all cats are animals, Tom is an animal, so Tom is a cat". The last step uses the converse, which isn't true.

If and only if, necessary, sufficient

RULE: if and only if

$P \Leftrightarrow Q$ is true when P and Q have the **same** value (both T or both F).

$P \Leftrightarrow Q \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$: the statement and its converse both hold.

It is the opposite of XOR: $P \Leftrightarrow Q \equiv \neg(P \oplus Q)$.

P	Q	$P \Leftrightarrow Q$
T	T	T
T	F	F
F	T	F
F	F	T

T when the two match,
F when they differ

RULE: the same "if" in different words

All of these mean $P \Rightarrow Q$: "if P then Q", "Q if P", "P **only if** Q", "P is **sufficient** for Q", "Q is **necessary** for P".

Only if points forward: "you can vote only if you are 18+" is vote $\Rightarrow$ 18+. Being 18 doesn't force you to vote.

Sufficient: P is enough to guarantee Q. **Necessary**: without Q, P can't happen (that is the contrapositive $\neg Q \Rightarrow \neg P$).

"Necessary **and** sufficient" means $P \Leftrightarrow Q$.

$\text{rain} \Rightarrow \text{clouds}$ ← *rain is sufficient for clouds; clouds are necessary for rain*

The same logic in Python

Programs are full of truth tables in disguise. Python spells the symbols out as words:

```
age = 20
has_ticket = False

if age >= 18 and has_ticket:
    print('Come in')
elif age >= 18 or has_ticket:
    print('Nearly!')
if not has_ticket:
    print('Buy a ticket')
```

age >= 18 is True (20 ≥ 18)
has_ticket is False

True and False → False: skip 'Come in'

True or False → True: prints 'Nearly!'

not False → True: prints 'Buy a ticket'

maths	$\neg$	$\wedge$	$\vee$	$\oplus$
Python	`not`	`and`	`or`	`^` `or` `!=`

Python bonus: "and" stops early if the left side is False (the answer must be False anyway). That's why "if x != 0 and 10 / x > 2:" never divides by zero.

"For all" and "there exists"

Two more symbols appear in proofs and in later maths courses:

$\forall$ *"for all" (an upside-down A, for All)*

$\exists$ *"there exists" (a backwards E, for Exists)*

$\forall x \in \mathbb{R},\ x^2 \geq 0$ reads "for every real number x, x squared is at least 0" (true). $\exists x \in \mathbb{Z},\ x + 5 = 2$ reads "there is an integer x with x + 5 = 2" (true: x = -3).

To prove a "for all" statement false, one counterexample is enough. "Every prime is odd"? 2 is prime and even. Done: false.

RULE: negating a quantifier

$\neg\,\forall x\, P(x) \equiv \exists x\, \neg P(x)$: "not all are" = "at least one isn't".

$\neg\,\exists x\, P(x) \equiv \forall x\, \neg P(x)$: "none is" = "every one isn't".

NOT swaps $\forall$ and $\exists$ and moves inside, like De Morgan swaps $\wedge$ and $\vee$.

$$\neg\,(\forall p \text{ prime},\ p \text{ odd}) \equiv \exists p \text{ prime},\ p \text{ even}$$ ← *witness: p = 2*

✗ Common mistakes

✗ $\neg\,(\text{all pass}) = \text{all fail}$

one failure is enough to make "all pass" false

✓ $\neg\,(\text{all pass}) = \text{some fail}$

Rules in this chapter

Learn these. Every worked example in the chapter is one of these rules applied step by step.

NOT, AND, OR	$\neg P$ flips; $A \wedge B$ T only if both T; $A \vee B$ F only if both F; $A \oplus B$ T if exactly one T
Rows	$\text{rows} = 2^n$ first letter TTTT FFFF, next TTFF TTFF, last TFTF TFTF
Precedence	$\neg$, then $\wedge$, then $\vee$, then $\Rightarrow$, then $\Leftrightarrow$; brackets first
Equivalent	same final column in every row ($\equiv$)
Tautology / contingency / contradiction	final column all T / mixed T and F / all F. Exams ask you to classify after the table.
De Morgan	$\neg(A \wedge B) = \neg A \vee \neg B$ and $\neg(A \vee B) = \neg A \wedge \neg B$
Implication	$P \Rightarrow Q \equiv \neg P \vee Q$ F only when P is T and Q is F
Negating an if	$\neg(P \Rightarrow Q) \equiv P \wedge \neg Q$
Converse, contrapositive	converse $Q \Rightarrow P$ (may be false); contrapositive $\neg Q \Rightarrow \neg P$ (always matches $P \Rightarrow Q$)
If and only if	$P \Leftrightarrow Q \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$ T when P and Q have the same value
Same "if", other words	P only if Q; P is sufficient for Q; Q is necessary for P: all mean $P \Rightarrow Q$
Quantifiers	$\neg\, \forall x\, P(x) \equiv \exists x\, \neg P(x)$ and $\neg\, \exists x\, P(x) \equiv \forall x\, \neg P(x)$; one counterexample disproves "for all"

Switchboard: your turn

Answers in grey. Draw the circuit or build the table for each one.

Statements

1. Is "x + 1 = 4" a statement? When?
2. Give the truth value of "3 > 5 or 2 < 4"
3. Give the truth value of "3 > 5 and 2 < 4"

1. only once x has a value (true if x = 3) 2. T 3. F

Tables

4. Build the truth table for ¬A ∧ B
5. Build the table for A ∨ ¬A. What do you notice?
6. Use a table to check ¬(A ∨ B) = ¬A ∧ ¬B

4. T only in the row A = F, B = T 5. always T (called a tautology) 6. the columns match

If... then

7. "If it rains, the match is off." It didn't rain and the match was off. Promise broken?
8. Write the converse and contrapositive of "if n is divisible by 4, n is even". Which are true?

7. no (row 4) 8. converse: if even then divisible by 4 (false: 6); contrapositive: if odd then not divisible by 4 (true)

Python

9. x = 5. Is (x > 3 and x < 10) True?
10. x = 5. Is not (x == 5 or x == 7) True?

9. True 10. False

Rules and bigger tables

11. How many rows does a table for P, Q, R and S need?
12. Write the R column of a P, Q, R table in the standard order
13. Put brackets into ¬P ∨ Q ∧ R to show the order
14. Rewrite P ⇒ Q using only ¬ and ∨
15. Negate "every bus is late"
16. "You can drive only if you have a licence." Write it as P ⇒ Q

11. $2^4 = 16$ 12. T F T F T F T F 13. (¬P) ∨ (Q ∧ R) 14. ¬P ∨ Q 15. some bus is not late 16. drive ⇒ licence (a licence is necessary, not sufficient)

Find the unknown

17. $P \Rightarrow Q$ is false. What are P and Q?
18. $P \vee Q$ is false. What are P and Q?
19. $P \Rightarrow Q$ is true and Q is false. What is P?
20. $P \Leftrightarrow Q$ is true and P is false. What is Q?
21. $P \wedge Q$ is true. What is $P \Rightarrow \neg Q$?
22. A truth table has 32 rows. How many letters are in it?

17. P = T, Q = F (the only F row) 18. both F (the only F row of OR) 19. F (P = T would make T ⇒ F, which is F) 20. F (⇔ needs them to match) 21. F (P = T, Q = T, so T ⇒ F) 22. $2^n = 32$, n = 5